

COMPILADORES

Análisis Léxico y Análisis Sintáctico



Informe Técnico

Analizador Léxico (Lexer) + Analizador Sintáctico (Parser)

Integrantes

Barreda Bejarano, Sebastian Raúl
Collado Cárdenas, Marco Antonio
Larota Ochoa, Samith Yowao
Larota Ochoa, Linkol Yowao

Curso: Compiladores | 2026

1. Introducción

Este proyecto consiste en la creación de un programa capaz de leer código escrito en un lenguaje de programación y transformarlo en una estructura que la computadora pueda procesar. El trabajo se divide en dos etapas principales: primero, identificar cada palabra y símbolo del código; segundo, revisar que el orden de esos elementos tenga sentido según las reglas que hemos definido.

2. Analizador Léxico (Lexer)

2.1 ¿Qué es el Análisis Léxico?

El analizador léxico es la primera fase del compilador. Su tarea es leer el código fuente carácter por carácter y agruparlos en unidades con significado llamadas TOKENS. Un token es la unidad mínima con sentido en el lenguaje (como una palabra en un idioma humano).

Por ejemplo, dado el código fuente:

```
int x = 10 + 5;
```

El analizador léxico produce los siguientes tokens:

Lexema	Tipo de Token
int	KEYWORD
x	IDENTIFIER
=	OPERATOR
10	LITERAL_INT
+	OPERATOR
5	LITERAL_INT
;	DELIMITER

2.2 Tokens Reconocidos

El lexer implementado reconoce los siguientes tipos de tokens:

Tipo	Descripción	Ejemplos
KEYWORD	Palabras reservadas del lenguaje	int, float, if, while, return
IDENTIFIER	Nombres de variables y funciones	x, suma, calcular
LITERAL_INT	Números enteros	0, 42, 100
LITERAL_FLOAT	Números de punto flotante	3.14, 0.5, 2.71
LITERAL_STRING	Cadenas de texto entre comillas	"hola", "mundo"
LITERAL_CHAR	Caracteres entre comillas simples	'a', '\n'
OPERATOR	Operadores aritméticos y relacionales	+, -, *, ==, !=, <=
DELIMITER	Delimitadores de estructura	;, (,), {, }

2.3 Mejoras Aplicadas al Lexer Original

Se realizaron las siguientes mejoras con respecto al lexer original:

- Soporte para números flotantes (3.14, 0.5) además de enteros.
- Soporte para literales de carácter ('a', '\n').
- Soporte para comentarios de bloque (`/* ... */`) multi-línea.
- Detección de operadores de dos caracteres: `==`, `!=`, `<=`, `>=`, `++`, `--`, `+=`, `-=`, `*=`, `/=`.
- Número de línea registrado en cada token para errores precisos.
- Reporte de errores con formato: Error lexico [línea N]: descripción.

3. Analizador Sintáctico (Parser)

3.1 ¿Qué es el Análisis Sintáctico?

El analizador sintáctico (parser) recibe la lista de tokens del lexer y verifica que su secuencia cumpla con las reglas gramaticales del lenguaje. Si la secuencia es correcta, construye un Árbol Sintáctico Abstracto (AST) que representa la estructura del programa.

Es análogo a la gramática de un idioma: así como en español 'el perro corre' es correcta pero 'perro el corre' no, el parser verifica que el código esté estructurado correctamente.

3.2 Tipo de Parser Implementado

Se implementó un parser Descendente Recursivo (Recursive Descent Parser), que es el tipo más didáctico y común para compiladores educativos. Sus características son:

- Top-Down: parte de la regla más general (programa) y desciende a las más específicas.
- Predictivo: mira el token actual para decidir qué regla aplicar (sin necesidad de retroceder).
- Cada regla gramatical tiene su propio método en el código C++.

3.3 Gramática Implementada

La gramática formal del lenguaje es la siguiente:

```
programa      → (declaracion_func | sentencia)*
sentencia     → declaracion | asignacion | bucle_for | si | mientras
               | bloque | retorno | llamada_func ";"
declaracion   → tipo IDENTIFIER ("=" expresion)? ";"
declaracion_f → tipo IDENTIFIER "(" parametros? ")" bloque
parametros    → tipo IDENTIFIER ("," tipo IDENTIFIER)*
asignacion    → IDENTIFIER "=" expresion ";"
si            → "if" "(" condicion ")" bloque ("else" bloque)?
mientras     → "while" "(" condicion ")" bloque
retorno       → "return" expresion ";"
llamada_func  → IDENTIFIER "(" (expresion ("," expresion)*)? ")"
condicion     → expresion op_rel expresion
expresion     → termino (("+"|"-" termino)*)
```

termino	→ factor (("*" "/" "%") factor)*
factor	→ NUMERO IDENTIFIER llamada_func "(" expresion ")"
tipo	→ "int" "float" "void"
op_rel	→ "==" "!=" "<" ">" "<=" ">="
bucle_for	→ "for" "(" (declaracion asignacion)? ";" condicion? ";" (asignacion incremento)? ")" bloque

3.4 Árbol Sintáctico Abstracto (AST)

El árbol sintáctico abstracto (AST) es la estructura de datos en la que el parser almacena el resultado del análisis. Cada nodo del árbol representa una construcción del lenguaje.

Por ejemplo, para la sentencia 'int suma = a + b;' el árbol generado es:

```
[Declaracion: int]
├── [Identificador: suma]
└── [Inicializacion]
    ├── [Suma: +]
    │   ├── [Identificador: a]
    │   └── [Identificador: b]
```

3.5 Construcciones Sintácticas Soportadas

- Declaración de variables con o sin inicialización: int x; / int y = 5;
- Asignación simple y compuesta: x = 10; / x += 5;
- Incremento y decremento: x++; / x--;
- Expresiones aritméticas con precedencia: a + b * c - d / e
- Sentencia if con o sin else: if (cond) { } else { }
- Sentencia while: while (cond) { }
- Llamadas a funciones con argumentos: calcular(x, y + 1)
- Sentencia return: return expr;
- Bloques anidados de código: { { } }
- Declaración de funciones: Soporte para definir funciones con tipo de retorno, nombre y lista de parámetros (ej. int suma(int x, int y) { ... }).
- Bucle for: Implementación de la estructura clásica de control for (inicialización; condición; incremento).

4. Ejemplos de Funcionamiento

Ejemplo 1: Declaraciones y Operaciones Aritméticas

Código de entrada (codigo.txt):

```
codigo.txt
1  int a = 10;
2  int b = 5;
3  float resultado = 3.14;
4  int suma = a + b;
5  int producto = a * b;
6  resultado = suma + producto;
```

Salida de parser:

=====

COMPILADOR DIDÁCTICO – LEXER + PARSER

=====

FASE 1: ANÁLISIS LÉXICO (LEXER)

► Tabla Léxica:

int	→ [KEYWORD]
a	→ [IDENTIFIER]
=	→ [OPERATOR]
10	→ [LITERAL_INT]
;	→ [DELIMITER]
int	→ [KEYWORD]
b	→ [IDENTIFIER]
=	→ [OPERATOR]
5	→ [LITERAL_INT]
;	→ [DELIMITER]
float	→ [KEYWORD]
resultado	→ [IDENTIFIER]
=	→ [OPERATOR]
3.14	→ [LITERAL_FLOAT]
;	→ [DELIMITER]
int	→ [KEYWORD]
suma	→ [IDENTIFIER]
=	→ [OPERATOR]
a	→ [IDENTIFIER]
+	→ [OPERATOR]
b	→ [IDENTIFIER]
;	→ [DELIMITER]
int	→ [KEYWORD]
producto	→ [IDENTIFIER]
=	→ [OPERATOR]
a	→ [IDENTIFIER]
*	→ [OPERATOR]

b	→ [IDENTIFIER]
;	→ [DELIMITER]
resultado	→ [IDENTIFIER]
=	→ [OPERATOR]
suma	→ [IDENTIFIER]
+	→ [OPERATOR]
producto	→ [IDENTIFIER]
;	→ [DELIMITER]

► Tabla de Símbolos:

```

a
b
resultado
suma
producto

```

FASE 2: ANÁLISIS SINTÁCTICO (PARSER)

✓ Análisis sintáctico exitoso. Sin errores.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol.

Los nodos Token_X representan terminales de la gramática.

```

[Programa]
├─ [Declaracion]
│   ├─ [Token_KEYWORD: int]
│   ├─ [Token_IDENTIFIER: a]
│   ├─ [Token_OPERATOR: =]
│   ├─ [Factor_LITERAL_INT: 10]
│   └─ [Token_DELIMITER: ;]
├─ [Declaracion]
│   ├─ [Token_KEYWORD: int]
│   ├─ [Token_IDENTIFIER: b]
│   ├─ [Token_OPERATOR: =]
│   ├─ [Factor_LITERAL_INT: 5]
│   └─ [Token_DELIMITER: ;]
└─ [Declaracion]
    ├─ [Token_KEYWORD: float]
    └─ [Token_IDENTIFIER: resultado]

```

```

|   └─ [Token_OPERATOR: =]
|   └─ [Factor_LITERAL_FLOAT: 3.14]
|   └─ [Token_DELIMITER: ;]
└─ [Declaracion]
|   └─ [Token_KEYWORD: int]
|   └─ [Token_IDENTIFIER: suma]
|   └─ [Token_OPERATOR: =]
|   └─ [Expresion_+: +]
|       └─ [Factor_ID: a]
|           └─ [Token_OPERATOR: +]
|               └─ [Factor_ID: b]
└─ [Token_DELIMITER: ;]
└─ [Declaracion]
|   └─ [Token_KEYWORD: int]
|   └─ [Token_IDENTIFIER: producto]
|   └─ [Token_OPERATOR: =]
|   └─ [Termino_*: *]
|       └─ [Factor_ID: a]
|           └─ [Token_OPERATOR: *]
|               └─ [Factor_ID: b]
└─ [Token_DELIMITER: ;]
└─ [Asignacion: =]
|   └─ [Token_IDENTIFIER: resultado]
|   └─ [Token_OPERATOR: =]
|   └─ [Expresion_+: +]
|       └─ [Factor_ID: suma]
|           └─ [Token_OPERATOR: +]
|               └─ [Factor_ID: producto]
└─ [Token_DELIMITER: ;]

```

► AST – Árbol Sintáctico Abstracto

Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

```

[Programa]
└─ [Declaracion: int]
|   └─ [Identificador: a]
|   └─ [Inicializacion]
|       └─ [NumeroEntero: 10]
└─ [Declaracion: int]
|   └─ [Identificador: b]
|   └─ [Inicializacion]
|       └─ [NumeroEntero: 5]

```

```

└─ [Declaracion: float]
|   └─ [Identificador: resultado]
|       └─ [Inicializacion]
|           └─ [NumeroFlotante: 3.14]
└─ [Declaracion: int]
|   └─ [Identificador: suma]
|       └─ [Inicializacion]
|           └─ [Suma: +]
|               └─ [Identificador: a]
|                   └─ [Identificador: b]
└─ [Declaracion: int]
|   └─ [Identificador: producto]
|       └─ [Inicializacion]
|           └─ [Multiplicacion: *]
|               └─ [Identificador: a]
|                   └─ [Identificador: b]
└─ [Asignacion: =]
    └─ [Identificador: resultado]
        └─ [Suma: +]
            └─ [Identificador: suma]
                └─ [Identificador: producto]

```

```

=====
RESUMEN DE COMPILACIÓN
=====

```

```

Tokens generados      : 35
Símbolos en tabla    : 5
Nodos raíz CST        : 6
Nodos raíz AST        : 6
Errores sintácticos   : 0
Estado                 : EXITOSO

```


FASE 1: ANÁLISIS LÉXICO (LEXER)

► Tabla Léxica:

int	→ [KEYWORD]
a	→ [IDENTIFIER]
=	→ [OPERATOR]
10	→ [LITERAL_INT]
;	→ [DELIMITER]
int	→ [KEYWORD]
b	→ [IDENTIFIER]
=	→ [OPERATOR]
5	→ [LITERAL_INT]
;	→ [DELIMITER]
float	→ [KEYWORD]
resultado	→ [IDENTIFIER]
=	→ [OPERATOR]
3.14	→ [LITERAL_FLOAT]
;	→ [DELIMITER]
int	→ [KEYWORD]
suma	→ [IDENTIFIER]
=	→ [OPERATOR]
a	→ [IDENTIFIER]
+	→ [OPERATOR]
b	→ [IDENTIFIER]
;	→ [DELIMITER]
int	→ [KEYWORD]
producto	→ [IDENTIFIER]
=	→ [OPERATOR]
a	→ [IDENTIFIER]
*	→ [OPERATOR]
b	→ [IDENTIFIER]
;	→ [DELIMITER]
resultado	→ [IDENTIFIER]
=	→ [OPERATOR]
suma	→ [IDENTIFIER]
+	→ [OPERATOR]
producto	→ [IDENTIFIER]
;	→ [DELIMITER]

► Tabla de Símbolos:

```
a
b
resultado
suma
producto
```

FASE 2: ANÁLISIS SINTÁCTICO (PARSER)

✓ Análisis sintáctico exitoso. Sin errores.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol. Los nodos Token_X representan terminales de la gramática.

```
[Programa]
├── [Declaracion]
│   ├── [Token_KEYWORD: int]
│   ├── [Token_IDENTIFIER: a]
│   ├── [Token_OPERATOR: =]
│   ├── [Factor_LITERAL_INT: 10]
│   └── [Token_DELIMITER: ;]
├── [Declaracion]
│   ├── [Token_KEYWORD: int]
│   ├── [Token_IDENTIFIER: b]
│   ├── [Token_OPERATOR: =]
│   ├── [Factor_LITERAL_INT: 5]
│   └── [Token_DELIMITER: ;]
├── [Declaracion]
│   ├── [Token_KEYWORD: float]
│   ├── [Token_IDENTIFIER: resultado]
│   ├── [Token_OPERATOR: =]
│   ├── [Factor_LITERAL_FLOAT: 3.14]
│   └── [Token_DELIMITER: ;]
└── [Declaracion]
    ├── [Token_KEYWORD: int]
    ├── [Token_IDENTIFIER: suma]
    ├── [Token_OPERATOR: =]
    ├── [Expresion_+: +]
    │   ├── [Factor_ID: a]
    │   ├── [Token_OPERATOR: +]
    │   └── [Factor_ID: b]
    └── [Token_DELIMITER: ;]
```

```

[Declaracion]
├── [Token_KEYWORD: int]
├── [Token_IDENTIFIER: producto]
├── [Token_OPERATOR: =]
├── [Termino *: *]
│   ├── [Factor_ID: a]
│   ├── [Token_OPERATOR: *]
│   └── [Factor_ID: b]
└── [Token_DELIMITER: ;]

[Asignacion: =]
├── [Token_IDENTIFIER: resultado]
├── [Token_OPERATOR: =]
├── [Expresion_+: +]
│   ├── [Factor_ID: suma]
│   ├── [Token_OPERATOR: +]
│   └── [Factor_ID: producto]
└── [Token_DELIMITER: ;]

```

- AST – Árbol Sintáctico Abstracto
 Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

```

[Programa]
├── [Declaracion: int]
│   ├── [Identificador: a]
│   └── [Inicializacion]
│       └── [NumeroEntero: 10]
├── [Declaracion: int]
│   ├── [Identificador: b]
│   └── [Inicializacion]
│       └── [NumeroEntero: 5]
├── [Declaracion: float]
│   ├── [Identificador: resultado]
│   └── [Inicializacion]
│       └── [NumeroFlotante: 3.14]
├── [Declaracion: int]
│   ├── [Identificador: suma]
│   └── [Inicializacion]
│       ├── [Suma: +]
│       ├── [Identificador: a]
│       └── [Identificador: b]
├── [Declaracion: int]
│   ├── [Identificador: producto]
│   └── [Inicializacion]
│       └── [Multiplicacion: *]

```

```

├── [Identificador: a]
├── [Identificador: b]
├── [Asignacion: =]
│   ├── [Identificador: resultado]
│   ├── [Suma: +]
│   ├── [Identificador: suma]
│   └── [Identificador: producto]

```

RESUMEN DE COMPILACIÓN

```

Tokens generados      : 35
Símbolos en tabla     : 5
Nodos raíz CST        : 6
Nodos raíz AST        : 6
Errores sintácticos   : 0
Estado                : EXITOSO

```

(PARA LOS SIGUIENTES EJEMPLOS OBTENDREMOS LAS CAPTURAS, ES DECIR, SOLO PEGAREMOS EL TEXTO DE LA SALIDA PARA EVITAR EL EXCESO DE ESPACIO NO UTILIZADO Y EVITAREMOS DE IGUAL FORMA LA PARTE DEL ANALIZADOR LÉXICO, PUES LA EVIDENCIA SE TRATA DEL ANALIZADOR SINTÁCTICO).

Ejemplo 2: Sentencias de Control (if/else y while)

Código de entrada (codigo.txt):

```
codigo.txt
1  int contador = 0;
2  int maximo = 5;
3  if (contador < maximo) {
4      acumulador = acumulador + contador;
5  }
6  while (contador < maximo) {
7      acumulador += contador;
8      contador++;
9  }
10 if (acumulador > 10) {
11     return 0;
12 } else {
13     return 1;
14 }
```

Salida Analizador Léxico (extracto):

```
|| FASE 2: ANÁLISIS SINTÁCTICO (PARSER) ||
```

✓ Análisis sintáctico exitoso. Sin errores.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol.

Los nodos Token_X representan terminales de la gramática.

[Programa]

├─ [Declaracion]

| ├─ [Token_KEYWORD: int]

| ├─ [Token_IDENTIFIER: contador]

| ├─ [Token_OPERATOR: =]

| ├─ [Factor_LITERAL_INT: 0]

| └─ [Token_DELIMITER: ;]

├─ [Declaracion]

| ├─ [Token_KEYWORD: int]

| ├─ [Token_IDENTIFIER: maximo]

| ├─ [Token_OPERATOR: =]

| ├─ [Factor_LITERAL_INT: 5]

| └─ [Token_DELIMITER: ;]

├─ [SentenciaIf]

| ├─ [Token_KEYWORD: if]

| └─ [Token_DELIMITER: (]

```

|   └─ [Condicion: <]
|   |   └─ [Factor_ID: contador]
|   |   └─ [Token_OPERATOR: <]
|   |   └─ [Factor_ID: maximo]
|   └─ [Token_DELIMITER: )]
|   └─ [Bloque]
|       └─ [Token_DELIMITER: {}]
|       └─ [Asignacion: =]
|           └─ [Token_IDENTIFIER: acumulador]
|           └─ [Token_OPERATOR: =]
|           └─ [Expresion_+: +]
|               └─ [Factor_ID: acumulador]
|               └─ [Token_OPERATOR: +]
|                   └─ [Factor_ID: contador]
|               └─ [Token_DELIMITER: ;]
|           └─ [Token_DELIMITER: {}]
└─ [SentenciaWhile]
    └─ [Token_KEYWORD: while]
    └─ [Token_DELIMITER: (]
    └─ [Condicion: <]
    |   └─ [Factor_ID: contador]
    |   └─ [Token_OPERATOR: <]
    |   └─ [Factor_ID: maximo]
    └─ [Token_DELIMITER: )]
    └─ [Bloque]
        └─ [Token_DELIMITER: {}]
        └─ [Asignacion: +=]
            └─ [Token_IDENTIFIER: acumulador]
            └─ [Token_OPERATOR: +=]
            └─ [Factor_ID: contador]
            └─ [Token_DELIMITER: ;]
        └─ [IncrDecr: ++]
            └─ [Token_IDENTIFIER: contador]
            └─ [Token_OPERATOR: ++]
            └─ [Token_DELIMITER: ;]
        └─ [Token_DELIMITER: {}]
└─ [SentenciaIf]
    └─ [Token_KEYWORD: if]
    └─ [Token_DELIMITER: (]
    └─ [Condicion: >]
    |   └─ [Factor_ID: acumulador]
    |   └─ [Token_OPERATOR: >]
    |   └─ [Factor_LITERAL_INT: 10]
    └─ [Token_DELIMITER: )]
    └─ [Bloque]

```

```

|   └─ [Token_DELIMITER: {}]
|   └─ [SentenciaReturn]
|   └─ [Token_KEYWORD: return]
|   └─ [Factor_LITERAL_INT: 0]
|   └─ [Token_DELIMITER: ;]
|   └─ [Token_DELIMITER: {}]
└─ [Token_KEYWORD: else]
└─ [Bloque]
    └─ [Token_DELIMITER: {}]
    └─ [SentenciaReturn]
    └─ [Token_KEYWORD: return]
    └─ [Factor_LITERAL_INT: 1]
    └─ [Token_DELIMITER: ;]
    └─ [Token_DELIMITER: {}]

```

► AST – Árbol Sintáctico Abstracto

Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

```

[Programa]
└─ [Declaracion: int]
|   └─ [Identificador: contador]
|   └─ [Inicializacion]
|       └─ [NumeroEntero: 0]
└─ [Declaracion: int]
|   └─ [Identificador: maximo]
|   └─ [Inicializacion]
|       └─ [NumeroEntero: 5]
└─ [SentenciaIf]
|   └─ [Condicion: <]
|   └─ [Identificador: contador]
|   └─ [Identificador: maximo]
|   └─ [Bloque]
|       └─ [Asignacion: =]
|           └─ [Identificador: acumulador]
|           └─ [Suma: +]
|               └─ [Identificador: acumulador]
|               └─ [Identificador: contador]
└─ [SentenciaWhile]
|   └─ [Condicion: <]
|   └─ [Identificador: contador]
|   └─ [Identificador: maximo]
|   └─ [Bloque]

```

```

|       |─ [AsignacionCompuesta: +=]
|       |   |─ [Identificador: acumulador]
|       |   |─ [Identificador: contador]
|       |─ [IncrementoDecremento: ++]
|       |   |─ [Identificador: contador]
|─ [SentenciaIf]
|   |─ [Condicion: >]
|   |   |─ [Identificador: acumulador]
|   |   |─ [NumeroEntero: 10]
|   |─ [Bloque]
|   |   |─ [SentenciaReturn]
|   |   |   |─ [NumeroEntero: 0]
|   |─ [Else]
|   |   |─ [Bloque]
|   |   |   |─ [SentenciaReturn]
|   |   |   |   |─ [NumeroEntero: 1]

```

RESUMEN DE COMPILACIÓN

```

Tokens generados      : 56
Símbolos en tabla    : 3
Nodos raíz CST        : 5
Nodos raíz AST        : 5
Errores sintácticos  : 0
Estado                : EXITOSO

```

Ejemplo 3: Llamadas a Funciones y Expresiones Compuestas

Código de entrada (código.txt):

```

código.txt
1  int x = 0;
2  x = calcular(10, 20);
3  y = sumar(x, 5);
4  z = multiplicar(x, y);
5  if (z != 0) { z = z - 1; }
6  int total = x + y + z;
7  return total;

```

Salida Analizador Sintáctico (extracto):

```

FASE 2: ANÁLISIS SINTÁCTICO (PARSER)

```

✓ Análisis sintáctico exitoso. Sin errores.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol.

Los nodos Token_X representan terminales de la gramática.

```
[Programa]
├── [Declaracion]
│   ├── [Token_KEYWORD: int]
│   ├── [Token_IDENTIFIER: x]
│   ├── [Token_OPERATOR: =]
│   ├── [Factor_LITERAL_INT: 0]
│   └── [Token_DELIMITER: ;]
├── [Asignacion: =]
│   ├── [Token_IDENTIFIER: x]
│   ├── [Token_OPERATOR: =]
│   ├── [LlamadaFuncion]
│   │   ├── [Token_IDENTIFIER: calcular]
│   │   ├── [Token_DELIMITER: (]
│   │   ├── [Factor_LITERAL_INT: 10]
│   │   ├── [Token_DELIMITER: ,]
│   │   ├── [Factor_LITERAL_INT: 20]
│   │   └── [Token_DELIMITER: )]
│   └── [Token_DELIMITER: ;]
├── [Asignacion: =]
│   ├── [Token_IDENTIFIER: y]
│   ├── [Token_OPERATOR: =]
│   ├── [LlamadaFuncion]
│   │   ├── [Token_IDENTIFIER: sumar]
│   │   ├── [Token_DELIMITER: (]
│   │   ├── [Factor_ID: x]
│   │   ├── [Token_DELIMITER: ,]
│   │   ├── [Factor_LITERAL_INT: 5]
│   │   └── [Token_DELIMITER: )]
│   └── [Token_DELIMITER: ;]
├── [Asignacion: =]
│   ├── [Token_IDENTIFIER: z]
│   ├── [Token_OPERATOR: =]
│   ├── [LlamadaFuncion]
│   │   ├── [Token_IDENTIFIER: multiplicar]
│   │   ├── [Token_DELIMITER: (]
│   │   ├── [Factor_ID: x]
│   │   ├── [Token_DELIMITER: ,]
│   │   ├── [Factor_ID: y]
│   │   └── [Token_DELIMITER: )]
│   └── [Token_DELIMITER: ;]
└── [SentenciaIf]
```



```

├─ [Token_KEYWORD: if]
├─ [Token_DELIMITER: (]
├─ [Condicion: !=]
│   └─ [Factor_ID: z]
│       └─ [Token_OPERATOR: !=]
│           └─ [Factor_LITERAL_INT: 0]
├─ [Token_DELIMITER: )]
└─ [Bloque]
    └─ [Token_DELIMITER: {]
        └─ [Asignacion: =]
            └─ [Token_IDENTIFIER: z]
                └─ [Token_OPERATOR: =]
                    └─ [Expresion_=: -]
                        └─ [Factor_ID: z]
                            └─ [Token_OPERATOR: -]
                                └─ [Factor_LITERAL_INT: 1]
                                    └─ [Token_DELIMITER: ;]
                                        └─ [Token_DELIMITER: }]
├─ [Declaracion]
│   └─ [Token_KEYWORD: int]
│   └─ [Token_IDENTIFIER: total]
│   └─ [Token_OPERATOR: =]
│   └─ [Expresion_+: +]
│       └─ [Expresion_+: +]
│           └─ [Factor_ID: x]
│               └─ [Token_OPERATOR: +]
│                   └─ [Factor_ID: y]
│                       └─ [Token_OPERATOR: +]
│                           └─ [Factor_ID: z]
└─ [Token_DELIMITER: ;]
└─ [SentenciaReturn]
    └─ [Token_KEYWORD: return]
    └─ [Factor_ID: total]
    └─ [Token_DELIMITER: ;]

```

- ▶ AST – Árbol Sintáctico Abstracto

Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

```
[Programa]
├─ [Declaracion: int]
│   ├─ [Identificador: x]
│   └─ [Inicializacion]
│       └─ [NumeroEntero: 0]
├─ [Asignacion: =]
│   └─ [Identificador: x]
```

```

└─ [LlamadaFuncion: calcular]
    └─ [NumeroEntero: 10]
        └─ [NumeroEntero: 20]
└─ [Asignacion: =]
    └─ [Identificador: y]
        └─ [LlamadaFuncion: sumar]
            └─ [Identificador: x]
                └─ [NumeroEntero: 5]
└─ [Asignacion: =]
    └─ [Identificador: z]
        └─ [LlamadaFuncion: multiplicar]
            └─ [Identificador: x]
                └─ [Identificador: y]
└─ [SentenciaIf]
    └─ [Condicion: !=]
        └─ [Identificador: z]
            └─ [NumeroEntero: 0]
        └─ [Bloque]
            └─ [Asignacion: =]
                └─ [Identificador: z]
                    └─ [Resta: -]
                        └─ [Identificador: z]
                            └─ [NumeroEntero: 1]
└─ [Declaracion: int]
    └─ [Identificador: total]
        └─ [Inicializacion]
            └─ [Suma: +]
                └─ [Suma: +]
                    └─ [Identificador: x]
                        └─ [Identificador: y]
                            └─ [Identificador: z]
└─ [SentenciaReturn]
    └─ [Identificador: total]

```

```

=====
RESUMEN DE COMPILACIÓN
=====

Tokens generados      : 58
Símbolos en tabla    : 7
Nodos raíz CST        : 7
Nodos raíz AST        : 7
Errores sintácticos   : 0
Estado                : EXITOSO

```

Ejemplo 4: Funciones y Bucles For

Código de entrada (codigo.txt):

```

≡ código.txt
1  int calcularFactorial(int n) {
2      int res = 1;
3      for (int i = 1; i <= n; i++) {
4          res = res * i;
5      }
6      return res;
7  }

```

Analisis Sintáctico (extracto):

FASE 2: ANÁLISIS SINTÁCTICO (PARSER)

✓ Análisis sintáctico exitoso. Sin errores.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol.

Los nodos Token_X representan terminales de la gramática.

```

[Programa]
├─ [DefinicionFuncion]
│   ├── [Token_KEYWORD: int]
│   ├── [Token_IDENTIFIER: calcularFactorial]
│   ├── [Token_DELIMITER: (]
│   ├── [Parametros]
│   │   └─ [Parametro]
│   │       ├── [Token_KEYWORD: int]
│   │       └─ [Token_IDENTIFIER: n]
│   ├── [Token_DELIMITER: )]
│   └─ [Bloque]
│       ├── [Token_DELIMITER: {]
│       ├── [Declaracion]
│       │   ├── [Token_KEYWORD: int]
│       │   ├── [Token_IDENTIFIER: res]
│       │   ├── [Token_OPERATOR: =]
│       │   ├── [Factor_LITERAL_INT: 1]
│       │   └─ [Token_DELIMITER: ;]
│       ├── [SentenciaFor]
│       │   ├── [Token_KEYWORD: for]
│       │   ├── [Token_DELIMITER: (]
│       │   ├── [For_Init]
│       │   │   └─ [Declaracion_Init]
│       │   │       ├── [Token_KEYWORD: int]
│       │   │       ├── [Token_IDENTIFIER: i]
│       │   │       ├── [Token_OPERATOR: =]
│       │   │       └─ [Factor_LITERAL_INT: 1]

```

```

|   └─ [Token_DELIMITER: ;]
|   └─ [For_Condicion]
|       └─ [Condicion: <=]
|           └─ [Factor_ID: i]
|           └─ [Token_OPERATOR: <=]
|               └─ [Factor_ID: n]
|   └─ [Token_DELIMITER: ;]
|   └─ [For_Paso]
|       └─ [IncrDecr_++: ++]
|           └─ [Token_IDENTIFIER: i]
|           └─ [Token_OPERATOR: ++]
|   └─ [Token_DELIMITER: )]
|   └─ [Bloque]
|       └─ [Token_DELIMITER: {}]
|       └─ [Asignacion: =]
|           └─ [Token_IDENTIFIER: res]
|           └─ [Token_OPERATOR: =]
|               └─ [Termino_*: *]
|                   └─ [Factor_ID: res]
|                   └─ [Token_OPERATOR: *]
|                       └─ [Factor_ID: i]
|                       └─ [Token_DELIMITER: ;]
|   └─ [Token_DELIMITER: {}]
└─ [SentenciaReturn]
    └─ [Token_KEYWORD: return]
    └─ [Factor_ID: res]
    └─ [Token_DELIMITER: ;]
└─ [Token_DELIMITER: {}]

```

► AST – Árbol Sintáctico Abstracto

Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

```

[Programa]
└─ [DefinicionFuncion: calcularFactorial]
    └─ [TipoRetorno: int]
    └─ [Parametros]
        └─ [Parametro: int]
            └─ [Identificador: n]
    └─ [Bloque]
        └─ [Declaracion: int]
            └─ [Identificador: res]
            └─ [Inicializacion]
                └─ [NumeroEntero: 1]
        └─ [SentenciaFor]
            └─ [For_Init]

```

```

┌─ [Declaracion: int]
│   ┌─ [Identificador: i]
│   │   ┌─ [Inicializacion]
│   │   │   ┌─ [NumeroEntero: 1]
│   └─ [For_Condicion]
│       ┌─ [Condicion: <=]
│       │   ┌─ [Identificador: i]
│       │   │   ┌─ [Identificador: n]
│       └─ [For_Paso]
│           ┌─ [IncrementoDecremento: ++]
│           │   ┌─ [Identificador: i]
│       └─ [Bloque]
│           ┌─ [Asignacion: =]
│           │   ┌─ [Identificador: res]
│           │   │   ┌─ [Multiplicacion: *]
│           │   │   │   ┌─ [Identificador: res]
│           │   │   │   │   ┌─ [Identificador: i]
└─ [SentenciaReturn]
    └─ [Identificador: res]

```

RESUMEN DE COMPILACIÓN

```

Tokens generados      : 38
Símbolos en tabla     : 4
Nodos raíz CST        : 1
Nodos raíz AST        : 1
Errores sintácticos   : 0
Estado                : EXITOSO

```

5. Detección de Errores

5.1 Errores Léxicos

El lexer detecta y reporta los siguientes errores:

- Carácter no válido: Error léxico [línea 3]: Caracter no valido '@'.
- Cadena de texto no cerrada: Error léxico [línea 5]: Cadena de texto no cerrada.
- Comentario de bloque no cerrado (sin el cierre */).
- Número flotante malformado (dos puntos decimales: 3.1.4).

EJEMPLO:

Entrada:

```

int calcularFactorial(int n) {
    int res = 1;
    for (int i = 1; i <= n; i++) {
        res = res * i;
    }
}

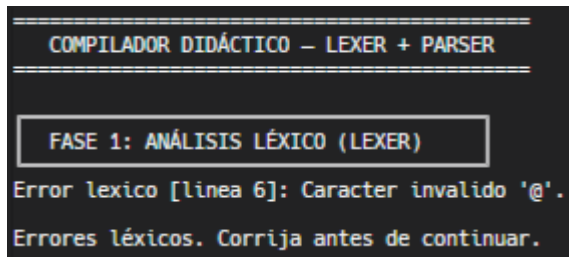
```

```

        return @res;
    }

```

Salida



5.2 Errores Sintácticos

El parser detecta y reporta los siguientes errores:

- Token inesperado: Error sintáctico [línea 4]: Se esperaba ';' pero se encontró 'int'.
- Factor inválido en expresión: Error sintáctico [línea 7]: Factor invalido '+'.
- Falta de paréntesis de cierre en condiciones o llamadas a función.

Cuando el parser encuentra un error, registra el mensaje y continúa el análisis para detectar más errores en la misma pasada.

EJEMPLO:

Entrada:

```

int x = 0;
x = calcular(10, 20);
y = sumar(x, 5);
z = multiplicar(x, y);
if (z != 0) { z = z - 1; }
int total = x + y + z
return total;

```

Salida:



► Errores Sintácticos:

Error sintactico [línea 8]: Se esperaba ';' pero se encontro 'return'.

► CST – Árbol de Análisis Concreto

Refleja EXACTAMENTE la gramática: contiene todos los tokens del código fuente como nodos hoja del árbol.

Los nodos Token_X representan terminales de la gramática.

```
[Programa]
├─ [Declaracion]
|   ├─ [Token_KEYWORD: int]
|   ├─ [Token_IDENTIFIER: x]
|   ├─ [Token_OPERATOR: =]
|   ├─ [Factor_LITERAL_INT: 0]
|   └─ [Token_DELIMITER: ;]
├─ [Asignacion: =]
|   ├─ [Token_IDENTIFIER: x]
|   ├─ [Token_OPERATOR: =]
|   └─ [LlamadaFuncion]
|       ├─ [Token_IDENTIFIER: calcular]
|       ├─ [Token_DELIMITER: (]
|       ├─ [Factor_LITERAL_INT: 10]
|       ├─ [Token_DELIMITER: ,]
|       ├─ [Factor_LITERAL_INT: 20]
|       └─ [Token_DELIMITER: )]
|   └─ [Token_DELIMITER: ;]
├─ [Asignacion: =]
|   ├─ [Token_IDENTIFIER: y]
|   ├─ [Token_OPERATOR: =]
|   └─ [LlamadaFuncion]
|       ├─ [Token_IDENTIFIER: sumar]
|       ├─ [Token_DELIMITER: (]
|       ├─ [Factor_ID: x]
|       ├─ [Token_DELIMITER: ,]
|       ├─ [Factor_LITERAL_INT: 5]
|       └─ [Token_DELIMITER: )]
|   └─ [Token_DELIMITER: ;]
├─ [Asignacion: =]
|   ├─ [Token_IDENTIFIER: z]
|   ├─ [Token_OPERATOR: =]
|   └─ [LlamadaFuncion]
|       ├─ [Token_IDENTIFIER: multiplicar]
|       ├─ [Token_DELIMITER: (]
|       ├─ [Factor_ID: x]
|       ├─ [Token_DELIMITER: ,]
|       ├─ [Factor_ID: y]
|       └─ [Token_DELIMITER: )]
|   └─ [Token_DELIMITER: ;]
└─ [SentenciaIf]
```

```

|   └─ [Token_KEYWORD: if]
|   └─ [Token_DELIMITER: (]
|   └─ [Condicion: !=]
|   |   └─ [Factor_ID: z]
|   |   └─ [Token_OPERATOR: !=]
|   |   └─ [Factor_LITERAL_INT: 0]
|   └─ [Token_DELIMITER: )]
|   └─ [Bloque]
|       └─ [Token_DELIMITER: {}]
|       └─ [Asignacion: =]
|       |   └─ [Token_IDENTIFIER: z]
|       |   └─ [Token_OPERATOR: =]
|       |   └─ [Expresion_=: -]
|       |       └─ [Factor_ID: z]
|       |       └─ [Token_OPERATOR: -]
|       |       └─ [Factor_LITERAL_INT: 1]
|       └─ [Token_DELIMITER: ;]
|       └─ [Token_DELIMITER: }]
└─ [Declaracion]
    └─ [Token_KEYWORD: int]
    └─ [Token_IDENTIFIER: total]
    └─ [Token_OPERATOR: =]
    └─ [Expresion_+: +]
    |   └─ [Expresion_+: +]
    |   |   └─ [Factor_ID: x]
    |   |   └─ [Token_OPERATOR: +]
    |   |   └─ [Factor_ID: y]
    |   └─ [Token_OPERATOR: +]
    |   └─ [Factor_ID: z]
    └─ [Error: return]
└─ [SentenciaReturn]
    └─ [Token_KEYWORD: return]
    └─ [Factor_ID: total]
    └─ [Token_DELIMITER: ;]

```

► AST – Árbol Sintáctico Abstracto

Versión simplificada del CST: elimina los nodos Token_X (paréntesis, ';', '{', '}', etc.) y conserva solo la estructura semántica relevante para el compilador.

[Programa]


```

├─ [Declaracion: int]
|   ├─ [Identificador: x]
|   └─ [Inicializacion]
|       └─ [NumeroEntero: 0]
├─ [Asignacion: =]
|   ├─ [Identificador: x]
|   └─ [LlamadaFuncion: calcular]
|       ├─ [NumeroEntero: 10]
|       └─ [NumeroEntero: 20]
├─ [Asignacion: =]
|   ├─ [Identificador: y]
|   └─ [LlamadaFuncion: sumar]
|       ├─ [Identificador: x]
|       └─ [NumeroEntero: 5]
├─ [Asignacion: =]
|   ├─ [Identificador: z]
|   └─ [LlamadaFuncion: multiplicar]
|       ├─ [Identificador: x]
|       └─ [Identificador: y]
├─ [SentenciaIf]
|   ├─ [Condicion: !=]
|   |   ├─ [Identificador: z]
|   |   └─ [NumeroEntero: 0]
|   └─ [Bloque]
|       └─ [Asignacion: =]
|           ├─ [Identificador: z]
|           └─ [Resta: -]
|               ├─ [Identificador: z]
|               └─ [NumeroEntero: 1]
├─ [Declaracion: int]
|   ├─ [Identificador: total]
|   └─ [Inicializacion]
|       └─ [Suma: +]
|           ├─ [Suma: +]
|           |   ├─ [Identificador: x]
|           |   └─ [Identificador: y]
|           └─ [Identificador: z]
└─ [SentenciaReturn]
    └─ [Identificador: total]

```

=====

RESUMEN DE COMPILACIÓN

```
=====
Tokens generados      : 57
Símbolos en tabla    : 7
Nodos raíz CST        : 7
Nodos raíz AST        : 7
Errores sintácticos   : 1
Estado                : CON ERRORES
```

6. Conclusiones

Tras el desarrollo de este proyecto, se han alcanzado los siguientes resultados clave:

- Traducción precisa del código: El sistema logra identificar y clasificar correctamente cada elemento del lenguaje, desde palabras clave y nombres de variables hasta números, textos y símbolos de operación.
- Validación de reglas: El programa no solo lee el código, sino que comprueba que esté escrito siguiendo el orden y la lógica correctos. Ahora admite estructuras más avanzadas como la creación de funciones propias y el uso de ciclos for para repetir tareas.
- Representación visual clara: Gracias a la creación de un árbol jerárquico, es posible visualizar exactamente cómo el compilador entiende la organización del programa, lo que ayuda a entender su estructura interna.
- Ayuda al usuario: El sistema es capaz de detectar fallos de escritura o de lógica e indicar exactamente en qué línea se encuentran, facilitando mucho la tarea de corregir el código fuente.
- Versatilidad demostrada: A través de los ejemplos prácticos, se ha comprobado que el software responde bien ante distintas situaciones, ya sea realizando cálculos matemáticos, gestionando decisiones lógicas o ejecutando procesos repetitivos.

En resumen, se ha construido una base sólida que permite procesar y entender un lenguaje de programación de manera eficiente, ordenada y modular.